IRanges - Basic Usage

Kasper D. Hansen

- Dependencies
- Corrections
- Overview
- Other Resources
- Basic IRanges
- Normal IRanges
- Disjoin
- Manipulating IRanges, intra-range
- Manipulating IRanges, as sets
- Finding Overlaps
- countOverlaps
- Finding nearest IRanges
- SessionInfo

Dependencies

This document has the following dependencies:

```
library(IRanges)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("IRanges"))
```

Corrections

Improvements and corrections to this document can be submitted on its GitHub in its repository.

Overview

A surprising amount of objects/tasks in computational biology can be formulated in terms of integer intervals, manipulation of integer intervals and overlap of integer intervals.

Objects: A transcript (a union of integer intervals), a collection of SNPs (intervals of width 1), transcription factor binding sites, a collection of aligned short reads.

Tasks: Which transcription factor binding sites hit the promoter of genes (overlap between two sets of intervals), which SNPs hit a collection of exons, which short reads hit a predetermined set of exons.

`IRanges` are collections of integer intervals. `GRanges` are like `IRanges`, but with an associated chromosome and strand, taking care of some book keeping.

Here we discuss `IRanges`, which provides the foundation for `GRanges`. This package implements (amongst other things) an algebra for handling integer intervals.

Other Resources

- The vignette titled “An Introduction to IRanges” from the IRanges webpage.

Basic IRanges

Specify IRanges by 2 of start, end, width (SEW).

```
ir1 <- IRanges(start = c(1,3,5), end = c(3,5,7))
ir1
```

```
## IRanges of length 3
##     start end width
## [1]     1   3     3
## [2]     3   5     3
## [3]     5   7     3
```

```
ir2 <- IRanges(start = c(1,3,5), width = 3)
all.equal(ir1, ir2)
```

```
## [1] TRUE
```

An `IRanges` consist of separate intervals; each interval is called a range. So `ir1` above contains 3 ranges.

Assessor methods: `start()`, `end()`, `width()` and also replacement methods.

```
start(ir1)
```

```
## [1] 1 3 5
```

```
width(ir2) <- 1
ir2
```

```
## IRanges of length 3
##     start end width
## [1]     1   1     1
## [2]     3   3     1
## [3]     5   5     1
```

They may have names

```
names(ir1) <- paste("A", 1:3, sep = "")
ir1
```

```
## IRanges of length 3
##     start end width names
## [1]     1   3     3    A1
## [2]     3   5     3    A2
## [3]     5   7     3    A3
```

They have a single dimension

```
dim(ir1)
```

```
## NULL
```

```
length(ir1)
```

```
## [1] 3
```

Because of this, subsetting works like a vector

```
ir1[1]
```

```
## IRanges of length 1
##     start end width names
## [1]     1   3     3    A1
```

```
ir1["A1"]
```

```
## IRanges of length 1
##     start end width names
## [1]     1   3     3    A1
```

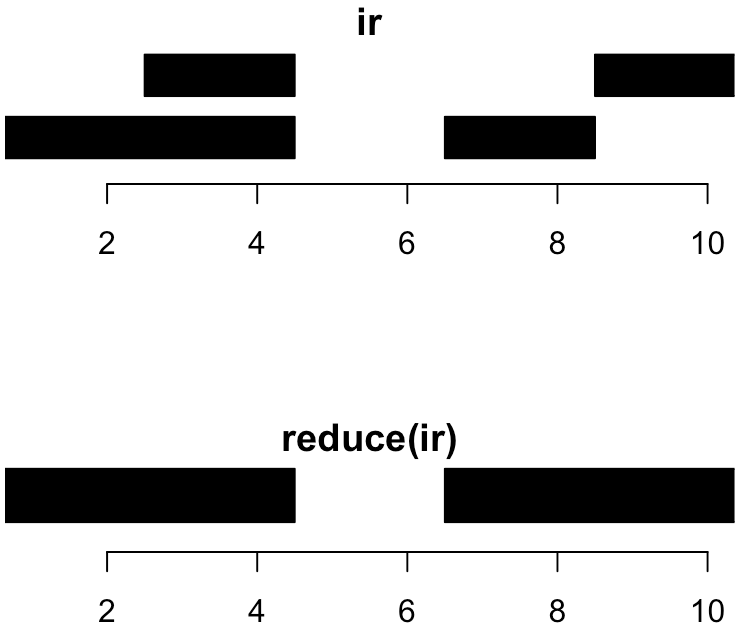
Like vectors, you can concatenate two `IRanges` with the `c()` function

```
c(ir1, ir2)
```

```
## IRanges of length 6
##     start end width names
## [1]     1   3     3    A1
## [2]     3   5     3    A2
## [3]     5   7     3    A3
## [4]     1   1     1
## [5]     3   3     1
## [6]     5   5     1
```

Normal IRanges

A normal IRanges is a minimal representation of the IRanges viewed as a set. Each integer only occur in a single range and there are as few ranges as possible. In addition, it is ordered. Many functions produce a normal `IRanges`. Created by `reduce()`.




```
ir
```

```
## IRanges of length 4
##     start end width
## [1]     1   4     4
## [2]     3   4     2
## [3]     7   8     2
## [4]     9  10     2
```

```
reduce(ir)
```

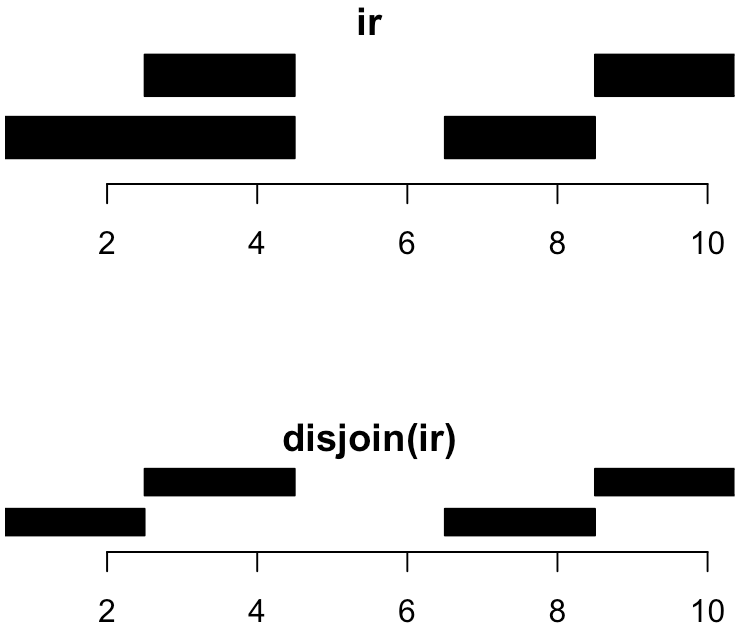
```
## IRanges of length 2
##     start end width
## [1]     1   4     4
## [2]     7  10     4
```

Answers: “Given a set of overlapping exons, which bases belong to an exon?”

Disjoin

From some perspective, `disjoin()` is the opposite of `reduce()`. An example explains better:

```
disjoin(ir1)
```



Answers: “Give a set of overlapping exons, which bases belong to the same set of exons?”

Manipulating IRanges, intra-range

“Intra-range” manipulations are manipulations where each original range gets mapped to a new range.

Examples of these are: `shift()`, `narrow()`, `flank()`, `resize()`, `restrict()`.

For example, `resize()` can be extremely useful. It has a `fix` argument controlling where the resizing occurs from. Use `fix="center"` to resize around the center of the ranges; I use this a lot.

```
resize(ir, width = 1, fix = "start")
```

```
## IRanges of length 4
##     start end width
## [1]     1   1     1
## [2]     3   3     1
## [3]     7   7     1
## [4]     9   9     1
```

```
resize(ir, width = 1, fix = "center")
```

```
## IRanges of length 4
##     start end width
## [1]     2   2     1
## [2]     3   3     1
## [3]     7   7     1
## [4]     9   9     1
```

The help page is `?"intra-range-methods"` (note that there is both a help page in *IRanges* and *GenomicRanges*).

Manipulating IRanges, as sets

Manipulating `IRanges` as sets means that we view each `IRanges` as a set of integers; individual integers is either contained in one or more ranges or they are not. This is equivalent to calling `reduce()` on the `IRanges` first.

Once this is done, we can use standard: `union()`, `intersect()`, `setdiff()`, `gaps()` between two `IRanges` (which all returns normalized `IRanges`).

```
ir1 <- IRanges(start = c(1, 3, 5), width = 1)
ir2 <- IRanges(start = c(4, 5, 6), width = 1)
union(ir1, ir2)
```

```
## IRanges of length 2
##     start end width
## [1]     1   1     1
## [2]     3   6     4
```

```
intersect(ir1, ir2)
```

```
## IRanges of length 1
##     start end width
## [1]     5   5     1
```

Because they return normalized `IRanges`, an alternative to `union()` is

```
reduce(c(ir1, ir2))
```

```
## IRanges of length 2
##     start end width
## [1]     1   1     1
## [2]     3   6     4
```

There is also an element-wise (pair-wise) version of these: `punion()`, `pintersect()`, `psetdiff()`, `pgap()`; this is similar to say `pmax` from base R. In my experience, these functions are seldom used.

Finding Overlaps

Finding (pairwise) overlaps between two `IRanges` is done by `findOverlaps()`. This function is very important and amazingly fast!

```
ir1 <- IRanges(start = c(1,4,8), end = c(3,7,10))
ir2 <- IRanges(start = c(3,4), width = 3)
ov <- findOverlaps(ir1, ir2)
ov
```

```
## Hits object with 3 hits and 0 metadata columns:
##       queryHits subjectHits
##       <integer>   <integer>
##   [1]         1           1
##   [2]         2           1
##   [3]         2           2
##   -------
##   queryLength: 3
##   subjectLength: 2
```

It returns a `Hits` object which describes the relationship between the two `IRanges`. This object is basically a two-column matrix of indicies into the two `IRanges`.

The two columns of the hits object can be accessed by `queryHits()` and `subjectHits()` (often used with `unique()`).

For example, the first row of the matrix describes that the first range of `ir1` overlaps with the first range of `ir2`. Or said differently, they have a non-empty intersection:

```
intersect(ir1[subjectHits(ov)[1]],
          ir2[queryHits(ov)[2]])
```

```
## IRanges of length 0
```

The elements of `unique(queryHits)` gives you the indices of the query ranges which actually had an overlap; you need `unique` because a query range may overlap multiple subject ranges.

```
queryHits(ov)
```

```
## [1] 1 2 2
```

```
unique(queryHits(ov))
```

```
## [1] 1 2
```

The list of arguments to `findOverlaps()` is long; there are a few hidden treasures here. For example, you can ask to only get an overlap if two ranges overlap by a certain number of bases.

```
args(findOverlaps)
```

```
## function (query, subject, maxgap = 0L, minoverlap = 1L, type = c("any
##     "start", "end", "within", "equal"), select = c("all", "first",
##     "last", "arbitrary"), algorithm = c("nclist", "intervaltree"),
##     ...)
## NULL
```

countOverlaps

For efficiency, there is also `countOverlaps()`, which just returns the number of overlaps. This function is faster and takes up less memory because it does not have to keep track of which ranges overlap, just the number of overlaps.

```
countOverlaps(ir1, ir2)
```

```
## [1] 1 2 0
```

Finding nearest IRanges

Sometimes you have two sets of `IRanges` and you need to know which ones are closest to each other. Functions for this include `nearest()`, `precede()`, `follow()`. Watch out for ties!

```
ir1
```

```
## IRanges of length 3
##     start end width
## [1]     1   3     3
## [2]     4   7     4
## [3]     8  10     3
```

```
ir2
```

```
## IRanges of length 2
##     start end width
## [1]     3   5     3
## [2]     4   6     3
```

```
nearest(ir1, ir2)
```

```
## [1] 1 1 2
```

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats4    parallel  methods   stats     graphics  grDevices utils
## [8] datasets  base
##
## other attached packages:
## [1] IRanges_2.2.7       S4Vectors_0.6.3     BiocGenerics_0.14.0
## [4] BiocStyle_1.6.0     rmarkdown_0.7
##
## loaded via a namespace (and not attached):
##  [1] magrittr_1.5    formatR_1.2     tools_3.2.1     htmltools_0.2.6
##  [5] yaml_2.1.13     stringi_0.5-5   knitr_1.11      stringr_1.0.0
##  [9] digest_0.6.8    evaluate_0.7.2
```